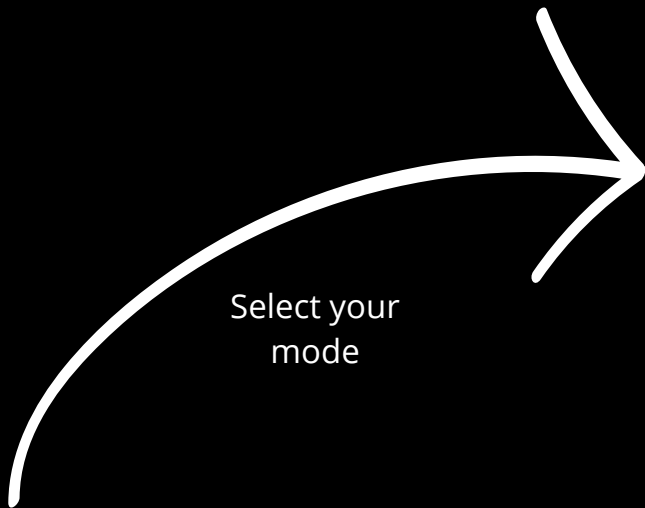


Game structure

```
Jungle_Game
├── env
│   └── ...
├── images
│   └── ...
├── game_optimized
│   ├── AI
│   │   ├── ai_minimax.py
│   │   ├── ai_vs_ai_main.py
│   │   ├── ai_vs_p_main.py
│   │   └── evaluate.py
│   ├── main_files
│   │   ├── board.py
│   │   ├── config.py
│   │   ├── game.py
│   │   ├── images.py
│   │   └── piece.py
│   ├── Player's
│   │   └── p_vs_p_main.py
│   ├── Random
│   │   ├── p_vs_rand_main.py
│   │   ├── rand_vs_rand_main.py
│   │   └── random_game.py
└── run_game_visual.py
```



Select your mode

python3 run_game_visual.py

The Jungle Game - Checkpoint

	A	B	C	D	E	F	G
9							
8							
7							
6							
5							
4							
3							
2							
1							

Next Player: 1

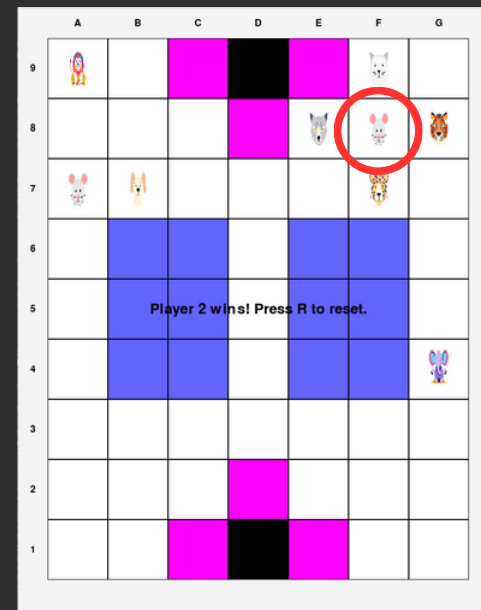
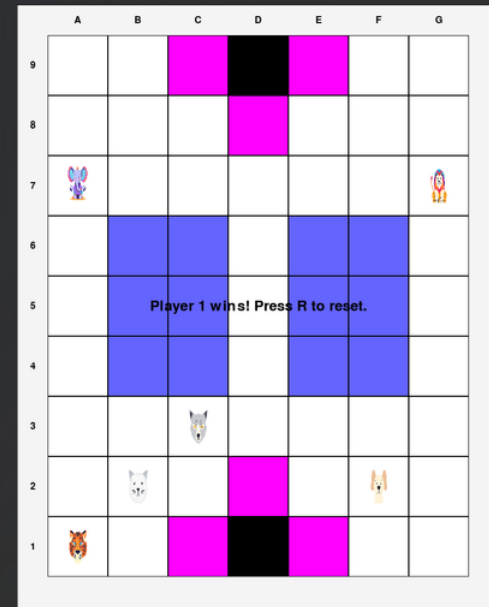
Initial state

```
def create_pieces(n):
    return [
        # Player 1 pieces
        Piece("tiger", 0, 8, 1),
        Piece("lion", 6, 8, 1),
        Piece("elephant", 0, 6, 1),
        Piece("dog", 5, 7, 1),
        Piece("wolf", 2, 6, 1),
        Piece("cat", 1, 7, 1),
        Piece("leopard", 4, 6, 1),
        Piece("mouse", 6, 6, 1),

        # Player 2 pieces
        Piece("lion", 0, 0, 2),
        Piece("tiger", 6, 0, 2),
        Piece("elephant", 6, 2, 2),
        Piece("dog", 1, 1, 2),
        Piece("wolf", 4, 2, 2),
        Piece("cat", 5, 1, 2),
        Piece("leopard", 2, 2, 2),
        Piece("mouse", 0, 2, 2)
    ]
```

Final state

n in [1,2]
 if player n in lair 3-n: player n wins
 elif no movement for player n: player 3-n wins
 elif player n pieces == 0: player 3-n wins
 else: no winner



Actions and results

Board and piece's rules

- The game is played on a grid (variable in the future) with specific board boundaries.
- Each player has a lair that cannot be entered by their own pieces but serves as the opponent's victory zone.
- Special trap cells weaken pieces, making them vulnerable to capture.
- Pieces move one square at a time in the four cardinal directions (horizontal and vertical - Fig.1).
- Only the mouse can enter water cells (Fig.2), and can't capture an animal exiting it; other pieces are restricted from water.
- Lions and tigers can jump over water (Fig.3) if no mouse blocks their path (Fig.4).
- A piece can capture an enemy by moving into its square if it meets the hierarchical strength rules, except a mouse can capture an elephant, and an elephant cannot capture a mouse.
- Capturing is automatic if an enemy piece occupies a cell (unless on a friendly piece or when restricted by water/movement rules).

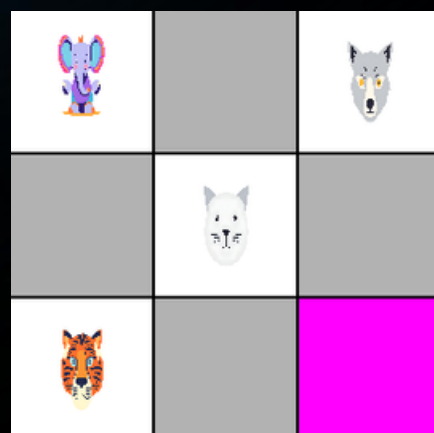


Fig.1

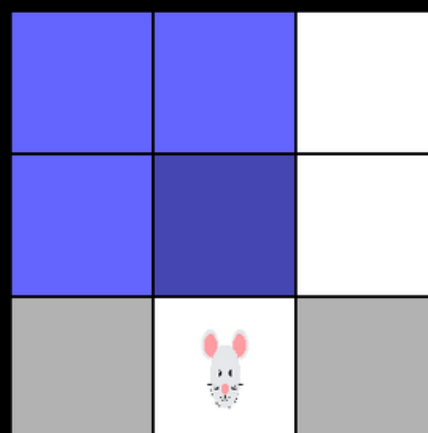


Fig.2

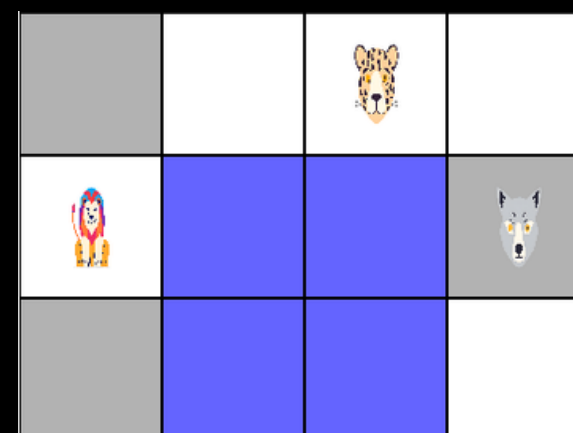


Fig.3

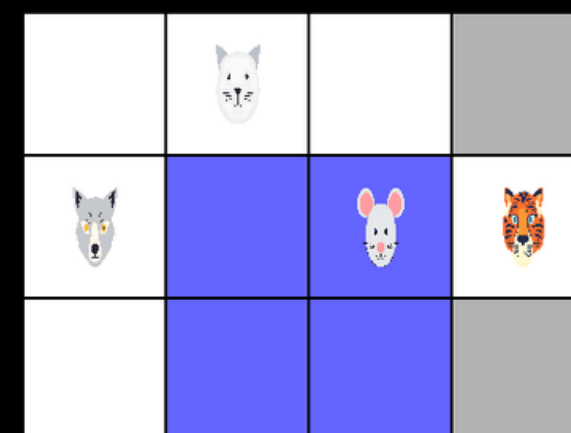


Fig.4

Minimax implementation

Cost for each movement is the same and equal to 1

Evaluate function

```
max_distance = 11  
factor_distance = 0.5  
factor_mobility = 0.2
```

Hierarchy evaluation

```
if (piece.x, piece.y) in opp_traps:  
    effective_value = 0  
if piece == "mouse": effective_value == 5  
else:  
    effective_value = Piece.hierarchy[piece.name]
```

Manhattan distance

```
if piece.player == player:  
    distance = abs(piece.x - opp_den[0]) + abs(piece.y - opp_den[1])  
    bonus_distance = (max_distance - distance) * factor_distance  
else:  
    distance = abs(piece.x - own_den[0]) + abs(piece.y - own_den[1])  
    bonus_distance = -(max_distance - distance) * factor_distance
```

Possible moves

```
mobility = len(game.get_valid_moves(piece))  
bonus_mobility = mobility * factor_mobility
```

Final_score* = effective_value + bonus_distance + bonus_mobility

Future

features/work

- Development of different board size's.
- Changing the animal images, to identify clearly the two teams.
- Develop new utility functions.
- Statistic work using different evaluate functions for minimax.
- Analyze time and space complexity.
- Improving the user interface.
- Add difficulty levels.
- Give hints to player in some mode's.
- Optional:
 - Add a simple tutorial.
 - Add a save/load mode, to save player statistics.
 - Use another AI methods such as negamax...

Bibliography

Minimax

Datacamp <https://www.datacamp.com/pt/tutorial/minimax-algorithm-for-ai-in-python>

Article <https://www.whhttps://www.comp.uems.br/~PFC/PFC%20179.pdfdatacamp.com/pt/tutorial/minimax-algorithm-for-ai-in-python>

Youtube
(tutorial) <https://www.youtube.com/watch?v=Bk9hINZc6sE>

Project developed by:
André Xie - up202407558
Duarte Gomes - up202409386
José Sousa - up202405046

AI'S

Chat <https://chatgpt.com>
gpt
Gemin <https://gemini.google.com>
i
zzzcode <https://zzzcode.ai/>

Game

Wikipedia [https://en.wikipedia.org/wiki/Jungle_\(board_game\)](https://en.wikipedia.org/wiki/Jungle_(board_game))

Brainking <https://gemini.https://brainking.com/en/GameRules?tp=56.com>